

Betriebsdokumentation

Aufbau, Konfiguration, Tests, Messungen, Protokollierung und Fehlerbehebung

Franziska Albrecht · Franziska.Sophie.Albrecht@proton.me

github.com/Franzer82/secure-agent-lab

Stand: 9. Oktober 2026

Für Betrieb und Übergabe an Projektteams. Alle Daten im Labor sind frei erfunden.

1. Zweck

Diese Dokumentation beschreibt, wie das Secure Agent Lab aufgebaut ist, wie man es installiert, die Tests und Messungen wiederholt, das Audit-Log liest und typische Fehler behebt. Sie ergänzt das Sicherheitskonzept (Zielarchitektur und Maßnahmen) und den Assessment-Bericht (Messergebnisse und Befunde).

Wichtig. Version 1 (`mcp_server/server_v1.py`, `agent/run_v1.py`) ist absichtlich verwundbar und darf nur lokal laufen, niemals öffentlich bereitgestellt werden. Das Labor arbeitet ausschließlich mit erfundenen Daten.

2. Aufbau des Repositorys

Verzeichnis	Inhalt
<code>agent/</code>	Agenten-Schleife (<code>agent_core.py</code>), Policies (<code>policy_v2.py</code> , <code>policy_v2_1.py</code>), Ausgabefilter, Konfiguration der Versionen (<code>versions.py</code>), Einstellungen, Tagesbudget, Hilfsskripte
<code>mcp_server/</code>	MCP-Server: <code>server_v1.py</code> (verwundbar), <code>server_v2.py</code> , <code>server_v2_1.py</code> sowie <code>create_lab_data.py</code> (erzeugt die erfundenen Daten)
<code>attacks/</code>	Szenarien (<code>scenarios.py</code> , <code>scenarios_a8s.py</code>), Runner, Wiedergabe, Auswertung, Inspektion
<code>lab_data/</code>	<code>public/</code> (lesbare Wissensdokumente) und <code>private/</code> (erfundene vertrauliche Dateien)
<code>tests/</code>	Alle Prüfungen ohne Modellzugriff sowie die Skripte <code>verify_baseline.py</code> und <code>verify_plan.py</code>
<code>docs/</code>	Bedrohungsmodell, Messpläne, Nachträge, diese Dokumentation
<code>reports/</code>	Zusammenfassungen. <code>reports/data/</code> enthält die Rohdaten mit Prüfsummen. <code>reports/raw/</code> (nicht im Git) enthält die laufenden Messdaten und das Tagesbudget
<code>outbox/</code>	Laufzeitdaten (nicht im Git): Tickets, simulierte Mails, Notizen, Audit-Log

2.1 Versionen und Konfigurationen

Name	Server	Policy	Notizen im Kontext	Zweck
v1	<code>server_v1</code>	<code>erlaubt alles</code>	Systemprompt	Ausgangslage, absichtlich verwundbar
v2	<code>server_v2</code>	<code>policy_v2</code>	Nutzer-Block, als Daten gekennzeichnet	Gehärtete Version, vollständig gemessen
v2n	<code>server_v2</code>	<code>policy_v2</code>	Systemprompt	Gegenprobe für A8s (nur die Notiz-Behandlung unterscheidet sich von v2)
v2.1	<code>server_v2_1</code>	<code>policy_v2_1</code>	Nutzer-Block, als Daten gekennzeichnet	Nachbesserung, nur durch Tests und Wiedergabe geprüft

Der Systemprompt von Version 2 und 2.1 ist der von Version 1 ohne die Zeile mit der Kennung. Er enthält bewusst keine zusätzlichen Sicherheitsanweisungen, damit sich die Wirkung der harten Kontrollen sauber zeigt.

3. Installation

Voraussetzung ist Python 3.13. Für die Tests ist kein API-Schlüssel nötig.

```
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt
python3 tests/run_all.py
```

Die Datei `requirements.txt` enthält `mcp>=2.3,<3`, `google-genai>=2.28,<3` und `python-dotenv`. Die Obergrenzen verhindern, dass eine neue Hauptversion unbemerkt Namen ändert (das MCP SDK 2 benannte FastMCP in MCPServer um).

Für Messungen mit dem Modell wird eine Datei `.env` im Projektordner gebraucht. Sie steht in der `.gitignore` und darf nie ins Repository.

```
GEMINI_API_KEY=hier_der_schluesse
```

4. Konfiguration

4.1 Umgebungsvariablen

Variable	Standard	Bedeutung
GEMINI_API_KEY	(keiner)	API-Schlüssel. Nur in der <code>.env</code> , nie im Code, nie in Ausgaben
GEMINI_MODEL	<code>gemini-3.5-flash-lite</code>	Feste Modellkennung. Keine Aliase wie „-latest“: Sie zeigen später auf ein anderes Modell und machen Messreihen unbrauchbar
GEMINI_MIN_INTERVAL	5.0	Mindestabstand zwischen zwei Anfragen in Sekunden
GEMINI_DAILY_BUDGET	400	Eigenes Tagesbudget. Maßgeblich bleibt das Dashboard von Google AI Studio

Zum Zeitpunkt der Messungen erlaubte die kostenlose Stufe für das verwendete Modell 15 Anfragen pro Minute, 250.000 Tokens pro Minute und 500 Anfragen pro Tag. Die Limits stehen im Dashboard und können sich ändern. Das Tagesbudget wird nach der Zeitzone UTC-8 gezählt.

4.2 Grenzwerte in Version 2.1

Regel	Wert	Stelle
Tickets je Gespräch	3	Policy
Tickets je Stunde (gleitend, global)	5	Server
Längen: Titel / Beschreibung / Betreff / Mailtext	120 / 1000 / 150 / 4000 Zeichen	Server
Notiz: Länge / Anzahl / Lebensdauer	200 Zeichen / 50 / 7 Tage	Server (Länge auch in der Policy)
Datei: maximal ausgegebene Zeichen	20000	Server
Audit-Log: maximale Länge eines Wertes	100 Zeichen	Server und Policy
Empfänger	genau eine Adresse der Domain <code>nordlicht-lab.example</code>	Server und Policy

Regeln, die in Policy und Server vorkommen, sind absichtlich doppelt vorhanden. `tests/test_policy_v2_1.py` vergleicht ihre Konstanten. Wer eine Regel ändert, ändert beide Stellen und ergänzt einen Test.

5. Tests

Alle Prüfungen laufen ohne Modellzugriff und sind bei jedem Lauf gleich. Das Sammelskript startet sie nacheinander:

```
python3 tests/run_all.py
python3 tests/verify_baseline.py
python3 tests/verify_plan.py
```

Datei	Prüft	Anzahl
test_scenarios.py	Auswertung der Szenarien und Kontrollfälle mit künstlichen Läufen, präparierte Dokumente	21
test_controls.py	Auswertung der Kontrollfälle K4 und K5	19
test_server_v2.py	Gehärteter Server (Version 2)	24
test_agent_v2.py	Policy, Ausgabefilter, Aufbau von Systemprompt und Nutzer-Block	44
test_versions.py	Zusammensetzung der Versionen v1 und v2	19
test_replay.py	Wiedergabe-Skript mit künstlichen Aufrufen	16
test_analyze.py	Auswertung des Sekundärmaßes	19
test_a8s.py	Nachtrag A8s und Konfiguration v2n	25
test_k6.py	Kontrollfall K6	9
test_server_v2_1.py	Server Version 2.1: Limits, Meldungen, Audit-Log	34
test_policy_v2_1.py	Policy Version 2.1, Audit-Log, Abgleich mit dem Server, Konfiguration v2.1	36
test_replay_v2_1.py	Wiedergabe mit Ziel Version 2.1	7
Gesamt		273

verify_baseline.py belegt, dass Szenarien, Auswertungen und Konstanten seit der Vorab-Festlegung inhaltlich unverändert sind. verify_plan.py belegt, dass der Messplan nur ergänzt wurde.

5.1 Pipeline

Die GitHub-Actions-Pipeline (.github/workflows/tests.yml) läuft bei jedem Push und Pull Request auf einem frischen Linux-Rechner, ohne Schlüssel. Sie prüft die Prüfsummen der Rohdaten, führt alle Tests aus, prüft Baseline und Messplan und durchsucht das Repository nach Schlüsselmustern. Die Historie wird vollständig ausgecheckt, weil die Prüfungen frühere Commits vergleichen. Dependabot schlägt wöchentlich Updates für die Actions und die Pakete vor, deren Pipeline-Lauf man vor dem Zusammenführen ansieht.

6. Messungen wiederholen

Messungen brauchen den API-Schlüssel und verbrauchen Kontingent. Der Runner zählt jede Anfrage, hält den Mindestabstand ein, speichert jeden Versuch sofort und setzt am nächsten Tag dort fort, wo er aufgehört hat.

Aufgabe	Befehl
Prüfen, ob Schlüssel und Modell erreichbar sind (eine Anfrage)	python3 agent/check_setup.py
Alle Szenarien mit Version 2, je 5 gültige Läufe	python3 attacks/runner.py --version v2 --runs 5
Nur bestimmte Szenarien (Groß-/Kleinschreibung egal)	python3 attacks/runner.py --version v2 --runs 5 --only K1,A1,A2b,A8s
Zusammenfassung neu erzeugen, ohne Anfrage	python3 attacks/runner.py --version v2 --summary-only

Aufgabe	Befehl
Ergebnisse sichern und neu beginnen	<code>python3 attacks/runner.py --version v2 --runs 5 --fresh</code>
Wiedergabe der Version-1-Aufrufe gegen Version 2.1 (ohne Modell)	<code>python3 attacks/replay_v1.py --target v2.1</code>
Sekundärmaß auswerten (ohne Modell)	<code>python3 attacks/analyze_v2.py</code>
Einen gespeicherten Versuch ansehen	<code>python3 attacks/inspect_trials.py A8s 1 v2</code>
Verfügbare Modelle auflisten	<code>python3 agent/list_models.py lite</code>
Tagesverbrauch und Budget ansehen	<code>python3 agent/usage_budget.py</code>
Laborzustand zurücksetzen	<code>python3 agent/lab_state.py</code>

6.1 Ablauf einer Messung

- 1 Plan schreiben: Kriterien, Vorhersagen und Entscheidungsregel. Plan committen, bevor gemessen wird.
- 2 Tests ausführen. Erst wenn alle grün sind, ist das Messinstrument geprüft.
- 3 Messen. Technische Fehler werden aussortiert und gezählt, sie zählen nie als „abgewehrt“.
- 4 Rohdaten mit Prüfsummen sichern (reports/data/), dann auswerten.
- 5 Ergebnisse an den Plan anhängen, nie den vorab festgelegten Teil ändern, und committen.

6.2 Abbruch und Fortsetzen

Der Runner bricht nach drei technischen Fehlern in Folge ab (Meldung ABBRUCH, Rückgabecode 2) und startet keinen Versuch, wenn das Tagesbudget nicht für einen ganzen Versuch reicht. Alle bisherigen Versuche sind gespeichert. Derselbe Befehl setzt später fort.

Gemessene Artefakte nicht ändern. Diese Dateien wurden gemessen und dürfen nicht verändert werden: attacks/scenarios.py, mcp_server/server_v1.py, mcp_server/server_v2.py, agent/policy_v2.py, agent/output_filter_v2.py. agent/agent_core.py wurde nach der Baseline erweitert, die Standardwerte für Version 1 sind unverändert (belegt durch Tests und die Messung von K4 und K5).

7. Audit-Log

Abgelehnte Aktionen werden in outbox/audit.jsonl protokolliert, eine JSON-Zeile je Ablehnung. Das Log enthält Ziele (Pfad, Empfänger) und Längen, nie Inhalte (keine Mailtexte, keine Notiztexte). Werte sind auf 100 Zeichen gekürzt. Zeilenumbrüche aus Modell-Eingaben können keine Logzeilen fälschen, weil jeder Wert als JSON geschrieben wird.

```
{"ts": "2026-10-09T12:04:11+00:00", "source": "server", "tool": "send_email",
  "code": "recipient_not_internal", "to": "spion@extern.example"}
```

Code	Quelle	Bedeutung
tool_not_allowed	Policy	Das Modell wollte ein nicht freigegebenes Werkzeug aufrufen
recipient_not_internal	Policy, Server	Mail an eine Adresse außerhalb der Firmendomain (oder mehrere Adressen)
path_not_allowed	Policy	Pfad mit „...“, absolut, mit Rückwärts-Schrägstrich oder Tilde
path_outside_public	Server	Aufgelöster Pfad liegt außerhalb des öffentlichen Bereichs

Code	Quelle	Bedeutung
invalid_path	Server	Pfad nicht auflösbar (etwa ungültiges Zeichen)
ticket_limit_run	Policy	Mehr als 3 Tickets in einem Gespräch
ticket_limit	Server	Mehr als 5 Tickets in der letzten Stunde
note_after_document_contact	Policy	Notiz nach dem Lesen von Dokumentinhalten im selben Gespräch
note_contains_address_or_link	Policy, Server	Notiz mit E-Mail-Adresse oder Link
note_empty / note_too_long	Policy, Server	Notiz leer oder länger als 200 Zeichen
memory_full	Server	Bereits 50 Notizen gespeichert
title_too_long, ...	Server	Titel, Beschreibung, Betreff oder Mailtext überschreitet das Längenlimit

7.1 Was ein Betreiber beobachten sollte

Das ist eine Empfehlung, umgesetzt ist nur das Schreiben des Logs:

- Häufung von `recipient_not_internal`, `path_outside_public` oder `tool_not_allowed` in kurzer Zeit: ein Hinweis auf Angriffsversuche oder eine Injektion.
- `ticket_limit` über längere Zeit: Das globale Limit ist ausgeschöpft, entweder Missbrauch oder zu knapp bemessen.
- Jeder Eintrag `note_*`: Jemand oder etwas versucht, das Gedächtnis zu beschreiben.

Grenzen. Das Log ist eine lokale Datei ohne Integritätsschutz, Aufbewahrungsregel und Alarmierung. Im Betrieb gehört es an ein zentrales System. Es enthält keine Nutzerkennung, weil das Labor keine kennt.

8. Fehlerbehebung

Symptom	Ursache	Maßnahme
429 RESOURCE_EXHAUSTED	Anfragekontingent (Minute oder Tag) erschöpft	Nicht wiederholen. Dashboard in AI Studio prüfen, <code>usage_budget.py</code> ansehen. Der Runner setzt später fort
ABBRUCH nach drei Fehlern in Folge	Meist Kontingent oder Netz	Meldung lesen, später denselben Befehl starten
Tagesbudget fast aufgebraucht	Eigenes Budget erreicht	Am nächsten Tag fortsetzen oder <code>GEMINI_DAILY_BUDGET</code> bewusst anheben
Code 404	Modellkennung unbekannt	<code>GEMINI_MODEL</code> in der <code>.env</code> prüfen, <code>list_models.py</code> nutzen
Code 401 / 403	Schlüssel ungültig	Schlüssel in AI Studio prüfen, neuen anlegen, die <code>.env</code> anpassen
Role 'tool' is not supported	Falsche Rolle für Werkzeugergebnisse	Der Code nutzt die Rolle „user“. Bei Änderungen am Agenten beibehalten
No module named <code>mcp.server.fastmcp</code>	MCP SDK in Version 2	<code>from mcp.server import MCPServer</code> verwenden, Obergrenze in den Abhängigkeiten beachten
<code>ExceptionGroup[...]</code>	Sammelfehler asynchroner Aufgaben	Den inneren Typ lesen. Der Runner nennt ihn in der Ausgabe
Unbekannte Szenarien	Kennung nicht in der Tabelle	Die Meldung nennt alle bekannten Kennungen

Symptom	Ursache	Maßnahme
shasum ... FAILED	Eine Rohdatei in reports/data/ wurde verändert	Nicht committen. Mit <code>git restore</code> wiederherstellen
verify_baseline.py meldet Unterschiede	Eine Szenario-Datei oder Auswertung wurde inhaltlich verändert	Änderung zurücknehmen oder offen im Plan dokumentieren
Pipeline rot	Test fehlgeschlagen oder Unterschied zwischen Mac und Linux	Log des fehlgeschlagenen Schritts lesen, lokal in einem frischen Klon nachstellen

9. Betrieb und Sicherheit

- **Schlüssel:** ein eigener Schlüssel je Projekt, nur in der `.env`, nie in Ausgaben. Bei Verdacht auf Abfluss den Schlüssel in AI Studio widerrufen und einen neuen anlegen.
- **Prüfung vor jeder Veröffentlichung:** Arbeitskopie und gesamte Historie auf Schlüsselmuster, lokale Pfade und fremde E-Mail-Adressen durchsuchen. Die Pipeline prüft Schlüsselmuster bei jedem Push.
- **Daten:** nur erfundene Daten. Die kostenlose Stufe der API kann Eingaben zur Produktverbesserung nutzen.
- **Version 1:** nur lokal. Der Laborzaun schützt die echte Umgebung, er ist keine Eigenschaft des Produkts.
- **Prozessumgebung:** Der Agent übergibt dem Server-Unterprozess ausdrücklich nur `PATH`. Dass der Prozess den Schlüssel nicht sieht, wurde nicht eigens getestet.
- **Modellwechsel:** Messungen gelten für das Modell, mit dem sie entstanden. Nach einem Wechsel die Tests ausführen und die Messreihe wiederholen, ohne Quoten verschiedener Modelle zu vermischen (der Runner zählt nur Versuche mit dem aktuellen Modell).

10. Übergabe-Checkliste

Prüfpunkt	Erledigt, wenn
Tests	<code>python3 tests/run_all.py</code> zeigt 273 von 273, Baseline und Messplan ohne Abweichung
Pipeline	Der letzte Lauf auf dem Hauptzweig ist grün
Schlüssel	Keine <code>.env</code> im Repository, Prüfung auf Schlüsselmuster ohne Treffer, Schlüssel wurde erneuert oder widerrufen
Rohdaten	Prüfsummen in reports/data/ stimmen
Dokumentation	Sicherheitskonzept, Assessment-Bericht und diese Dokumentation liegen in docs/
Restrisiken	Die Liste im Sicherheitskonzept (Abschnitt 9) wurde gelesen und dem Verantwortlichen übergeben
Empfehlungen	Identität und Rollen, Freigabeprozess, zentrales Log: Entscheidung dokumentiert, ob und wann umgesetzt

11. Glossar

Begriff	Bedeutung
MCP (Model Context Protocol)	Protokoll, über das ein Modell Werkzeuge eines Servers nutzt
Prompt Injection	Eine Anweisung in Eingaben oder Inhalten, die das Modell als Befehl behandelt. Direkt (vom Nutzer) oder indirekt (aus einem Dokument)

Begriff	Bedeutung
Excessive Agency	Der Agent hat mehr Funktionen, Rechte oder Autonomie, als die Aufgabe braucht (OWASP LLM06)
Policy	Prüfung jedes Werkzeugwunsches außerhalb des Modells
Canary-Marker	Eindeutige Kennung in vertraulichen Daten, an der ein Abfluss automatisch erkannt wird
Fail-closed	Bei beschädigten oder unklaren Daten wird sicher abgelehnt, nicht erlaubt
Mehrstufige Verteidigung	Dieselbe Regel gilt an mehreren Stellen, damit der Ausfall einer Stelle nicht genügt
Bodeneffekt	Ein Angriff, der schon vorher nie gelang, kann durch eine Kontrolle nicht weiter verbessert werden
Wiedergabe	Gespeicherte Werkzeugaufrufe aus Version 1 werden gegen eine spätere Version abgespielt, ohne Modell